System Documentation

for

SR Automate System

Version 1.0

Created By : Raziq Din bin Mohd Tasriri

Date: **18 February 2026**

Contents

Contents..... 2
Revisions..... 4
1 Project Management..... 5
1.1 Team Members..... 5
1.2 Problem statement..... 5
1.3 Project Plan..... 5
2 System Overview.....6
2.1 Description..... 6
2.2 Tasks..... 6
2.3 Modules Developed..... 6
2.4 Use Case Diagram..... 8
3 Requirements..... 9
3.1 Class Diagrams / ERD..... 9
3.2 State Diagrams..... 10
4 Design..... 11
4.1 Data Dictionary..... 11
4.1.1 Data Structures..... 11
4.2 Software Architecture..... 14
4.3 Main Screens..... 20
4.3.1 Class Diagram Generation Page..... 20
4.3.2 Flowchart Generation Page..... 21
4.3.3 View Recent Diagrams..... 21
4.4 Main Components..... 22
4.4.1 Generate Diagrams..... 23
4.4.2 Edit Diagram Code..... 24
4.4.3 View Recent Diagrams..... 25
5 Implementation Details..... 26
5.1 Development Environment..... 26
5.2 Software Integration..... 26
5.3 Database..... 26
6 Testing..... 27
6.1 Testing Strategy..... 27
6.2 Test Data..... 27
6.3 Acceptance Testing..... 27
7 Sample Screens..... 28
7.1 Sequence Diagram Generation Page..... 28

7.1.1 Class Diagram Generation Page..... 29
7.1.2 Flowchart Generation Page.....30
7.1.3 View Recent Diagrams..... 31
7.1.4 Edit Diagram Page..... 32
8 User Guide..... 33
9 Conclusion..... 34
References..... 35

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
SRS in Part 1(as Ver 1.0) SDS in Part 2(as Ver 2.0.X) *System Documentation in Part 3 (as Ver 3.0) Draft Type and Number	Raziq Din bin Mohd Tasriri	Information about the revision. This table does not need to be filled in whenever a document is touched, only when the version is being upgraded.	00/00/00

1 Project Management

1.1 Team Members

Name	Actor
Raziq Din bin Mohd Tasriri	User

1.2 Problem statement

Currently, the process of documenting Service Requests (SRs) and system workflows is a manual, time-consuming task. Staff and users often struggle to translate complex technical requirements into clear visual diagrams. Existing tools require manual drag-and-drop actions or specialized knowledge of diagramming syntax (like Mermaid JS), leading to inconsistencies, outdated documentation, and delays in request fulfillment. There is a clear need for an automated system that can interpret natural language and instantly generate accurate visual representations.

The primary purpose of this document is to define the functional and non-functional requirements for the **Software Requirements Automate (SR-Automate)** system. SR-Automate is designed to leverage automation to streamline the design phase of the Software Development Life Cycle (SDLC). By automating the generation and organization of architectural diagrams and documentation, this software aims to reduce manual overhead, minimize human error in technical drafting, and significantly enhance developer productivity.

1.3 Project Plan

2 System Overview

2.1 Description

Actors	Major Processes
User	Enter user prompts and generate Visual Diagrams

2.2 Tasks

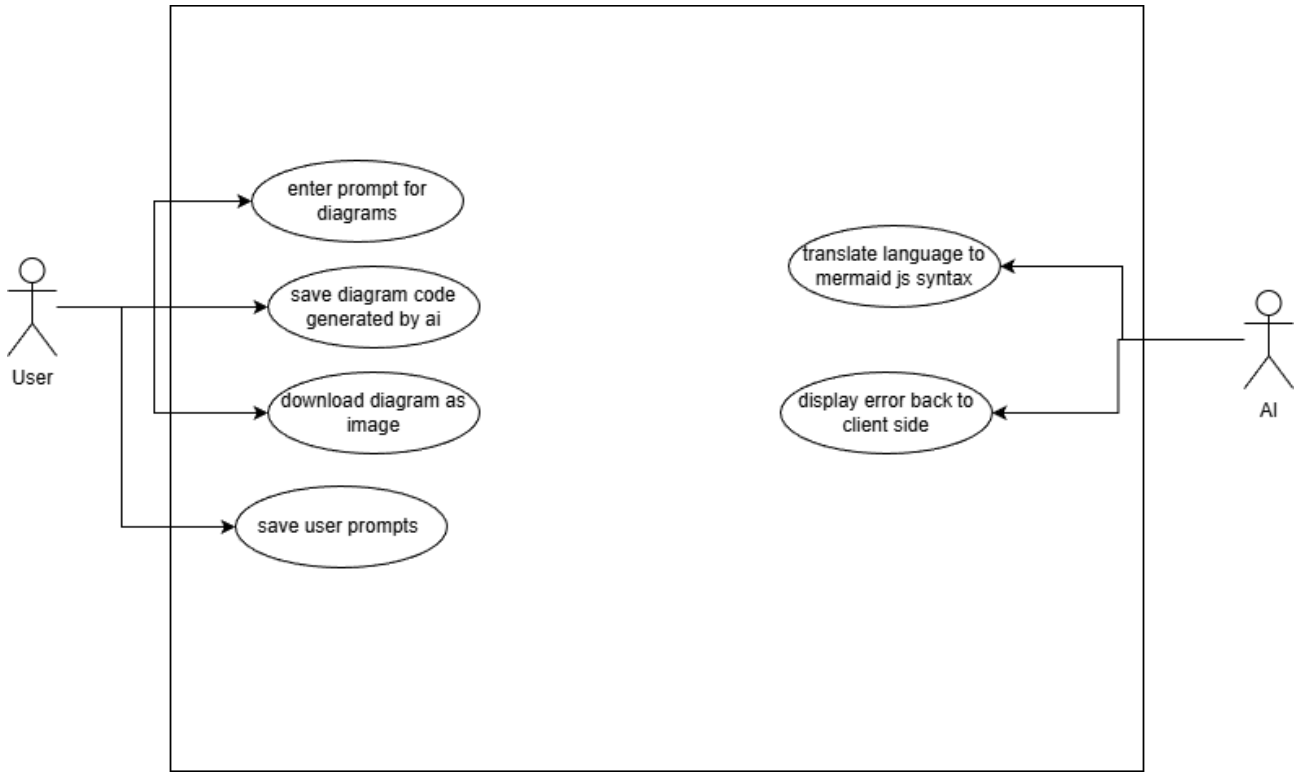
Team Member	Tasks
Raziq	Documents Software Requirement Specifications
	Development
	Testing

2.3 Modules Developed

Team member	Modules Developed	Assumptions
Raziq	AI Orchestration Module	It is assumed that the hosting environment (or local machine) has sufficient hardware resources (GPU/RAM) to run Ollama and the Gemma 3 model with acceptable latency.
	Database Schema & Integration	Assumes that the Pocketbase instance is persistently reachable via API calls for real-time saving and fetching of diagrams.
	Diagram Retrieval Engine	It is assumed that the AI model will provide syntactically correct Mermaid JS code; if the syntax is broken, the rendering library (Mermaid JS) will handle the error gracefully without crashing the site.
	Single User Authentication	For the current version, it is assumed that "Client IDs" are unique enough to prevent data

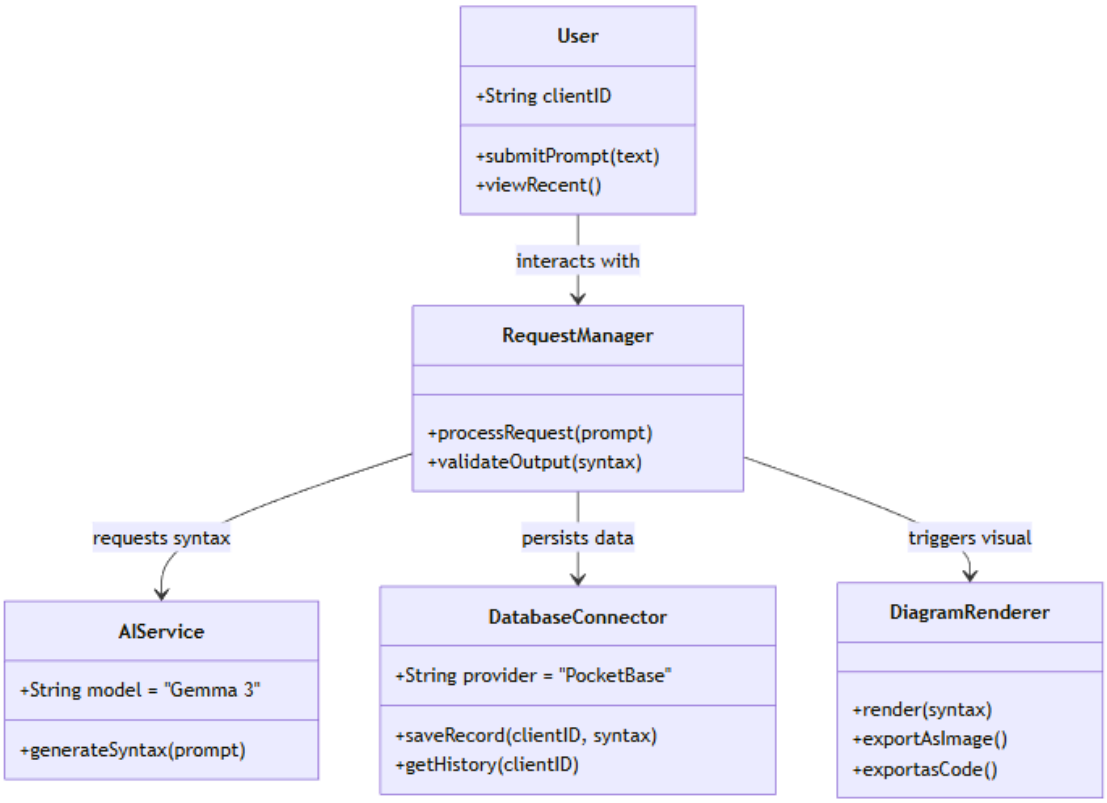
		leakage between different users during the fetching process.

2.4 Use Case Diagram

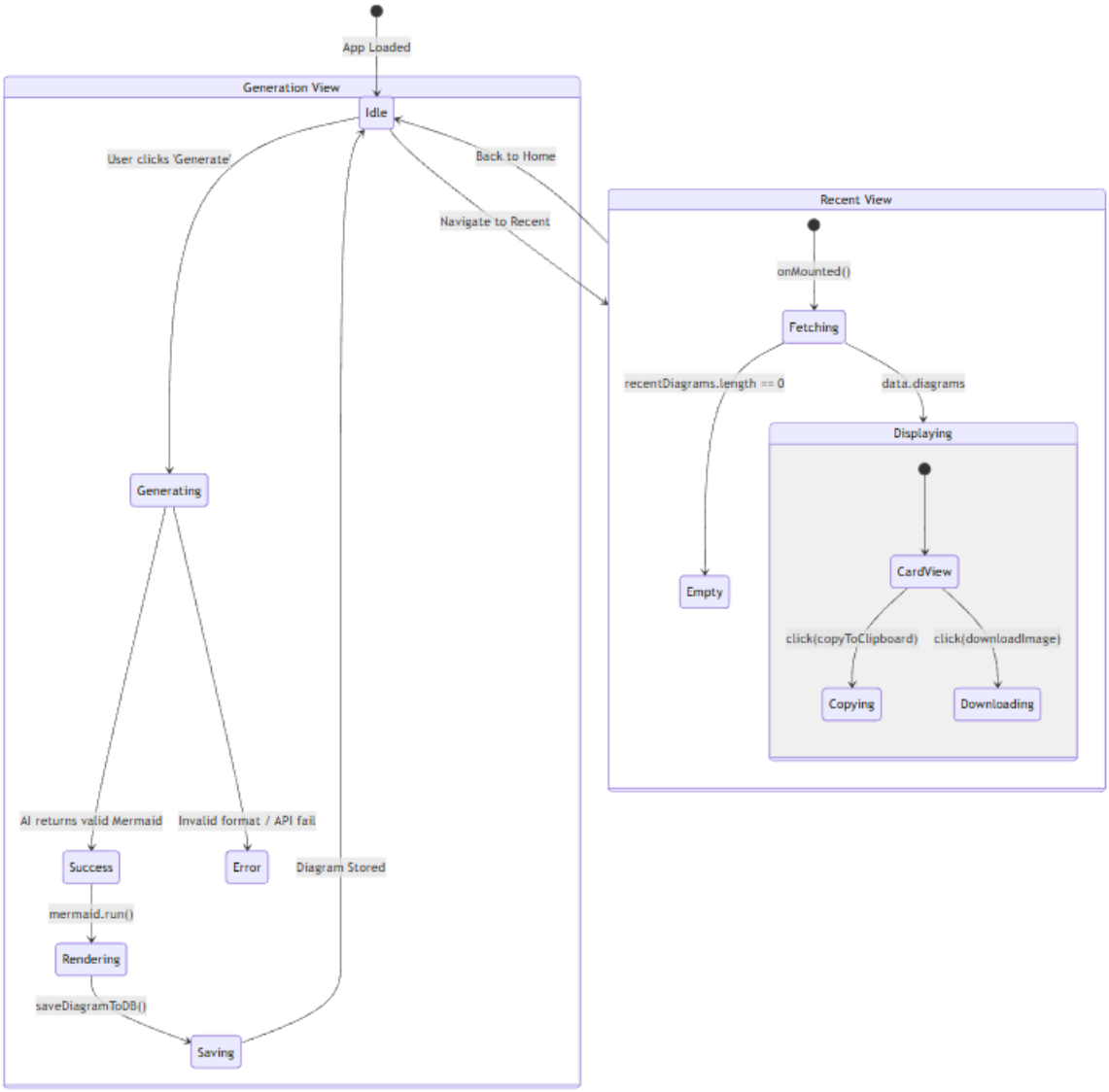


3 Requirements

3.1 Class Diagrams / ERD



3.2 State Diagrams



4 Design

4.1 Data Dictionary

<TO DO: Describe the data dictionary and place the table with the details here>

Table name	*Data attributes*	*Data types*	*Data key value*	*Data dependency*	*Data requirements*
Users	*User_id*	*Integer*	*PK*	*-*	*Yes*
	Session_id	*Text*	*-*	*-*	*Yes*
Diagrams	*Diagram_id*	*Integer*	*PK*	*-*	*Yes*
	Diagram_prompt	*Text*	*-*	*-*	*Yes*
	Diagram_png	*blob*	*-*	*-*	*Yes*
	Diagram_code	*Text*	*-*	*-*	*Yes*
	Diagram_type	*Text*	*-*	*-*	*Yes*
	User_id	*Text*	*FK*	*Users*	*Yes*

4.1.1 Data Structures

4.1.1.1 Users

This table is used to store user and unique generated session id of users from local storage , this is used to identify different users without require them to sign in

Data attributes	*Purpose*
User_id	To uniquely identify each users that is recorded in the application
Session_id	To store each client id that is generated based on local storage id of each devices

Example data :

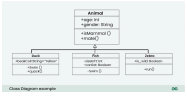
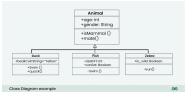
User_id	*Session_id*
1	eb1_1ashdhsaksdAhsa
2	Hje_12sjaskalsjd

4.1.1.1 Diagram

This table is used to store user prompts , diagram generated , type of diagrams , diagram source code to be stored in according to each user's session id

Data attributes	*Purpose*
Diagram_id	To uniquely identify each diagrams that is recorded in the application
Diagram_prompt	Store user prompts based on successful generation
Diagram_png	Store diagrams diagram in form of svg format
Diagram_code	Store diagram source code based on Mermaid JS syntax
Diagram_type	Store diagram type that the user generated based on the type of diagrams user is generating
User_id	To store each client id that is generated based on local storage id of each devices

Example data :

Diagram_id	Diagram_prompt	Diagram_png	Diagram_code	Diagram_type	User_id
1	Create a class diagram for …		ClassDiagram = { User = …	Class Diagram	1
2	Create a sequence diagram for …		SequenceDiagram = { User = …	Sequence Diagram	2
3	Create a sequence diagram for …		SequenceDiagram = { User = …	Sequence Diagram	1

4.2 Software Architecture

High Level Architecture

This diagram offers a high-level overview of SR Automate's architecture. It visually displays the system's structure and the key components involved. Detail descriptions of the main system architecture are displayed below. The system is divided into four components, which are client-side, frontend, server-side and data layer. The client-side consists of user devices that interact with the web application itself. The frontend consists of this project's software which servers the main functionalities and solving the problem statements from the stakeholders. The server-side, also known as backend, consists of Web API and Services, which communicates with the data layer of the application. The data layer consists of databases and folders that will store the data and user's files on the server. This diagram shows a clear explanation of how the architecture is broken down in visuals.

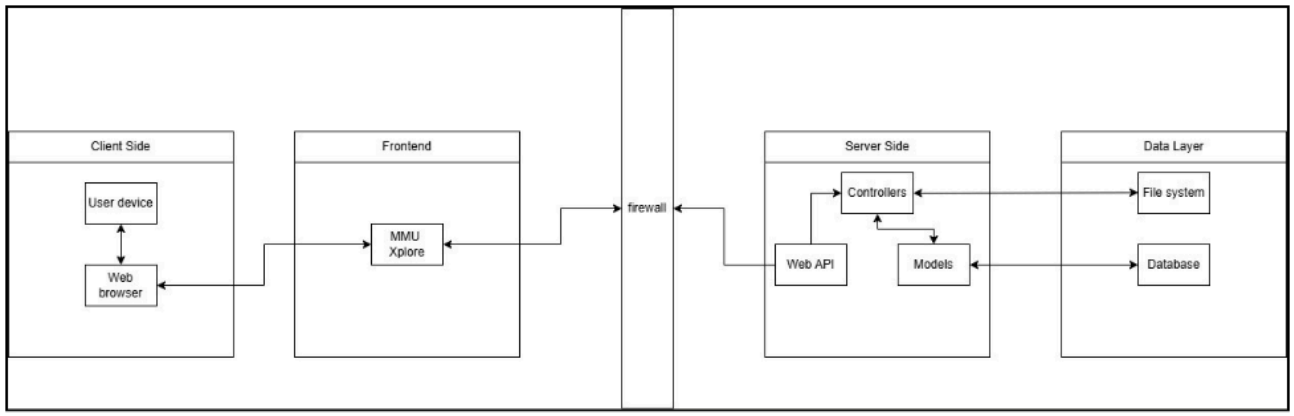


Figure 4.2.1: Higher-Level Architecture Diagram

Frontend Architecture

The frontend architecture consists of a single client-side section, both user device and web browser is included in the client-side section and the frontend, which includes the components required to build the interface of the website and enhance user experience of the website. On the user device, there is only User that will access the system through a web browser, sending http requests and receiving webpages as components back to their device. The web browser which has the website, consists of a presentation layer, user interface and container, which includes essential components like sidebar, cards , forms, buttons, text input and field area. These components are interactable with the pages, catering smooth communication with the backend to ensure a cohesive and efficient user experience to the user of different actors.

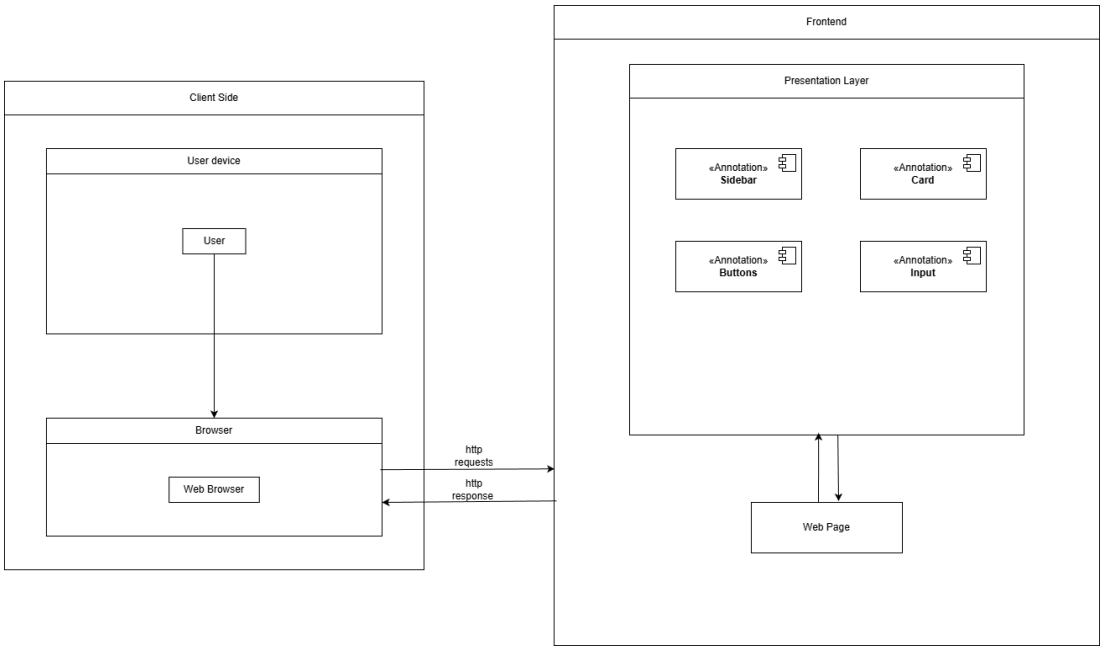


Figure 4.2.1: Frontend Architecture Diagram

Backend Architecture

The backend, which is called server side from the architecture diagram, is divided into several layers and components, ensuring a structured and modular approach to manage different actors and features on the website. At the primary requirements, the system consists of business layers, which consists of Models and Controller components. The Controller handles different functions held by the User with the database of the project. Models correspond to these actors, ensuring a clean and clear data handling and processing. The system features a Web API section, that includes a backend API and Web Server, to enable communications between both frontend and backend applications. This architecture effectively manages data flow and interactions in the system, providing a flexible and robust framework for the website's entire operations and features.

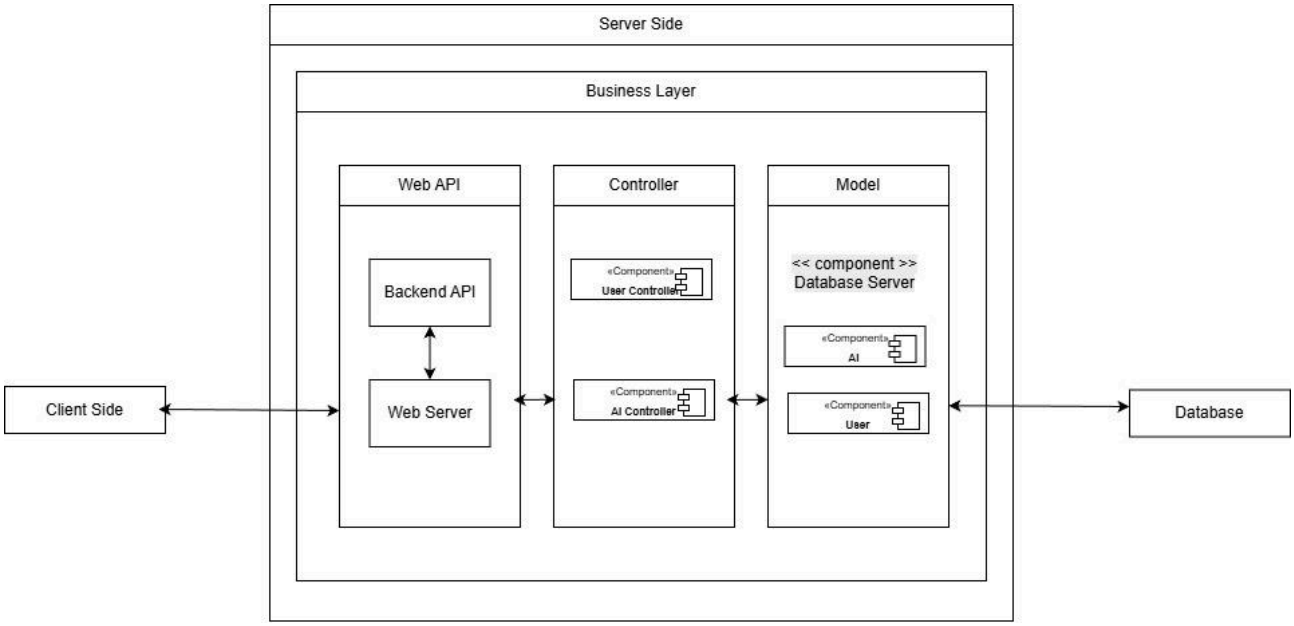


Figure 4.2.1: Backend Architecture Diagram

Data Layer Architecture

The data layer architecture is created to efficiently manage and handle data storage. It consists of two primary containers. The first container contains Pocketbase , that is built on SQlite and a dedicated web server to handle database operations and webserver-based data transactions. The second container contains File storage with another dedicated webserver, managing file storage, retrieval and insertion in a dedicated folder. Both containers are interactable with the server side (backend), ensuring smooth data operations and accessibility. This architecture highlights the system's robust and scalable approach to data management, providing a solid support for different actors and its functionalities on the client side.

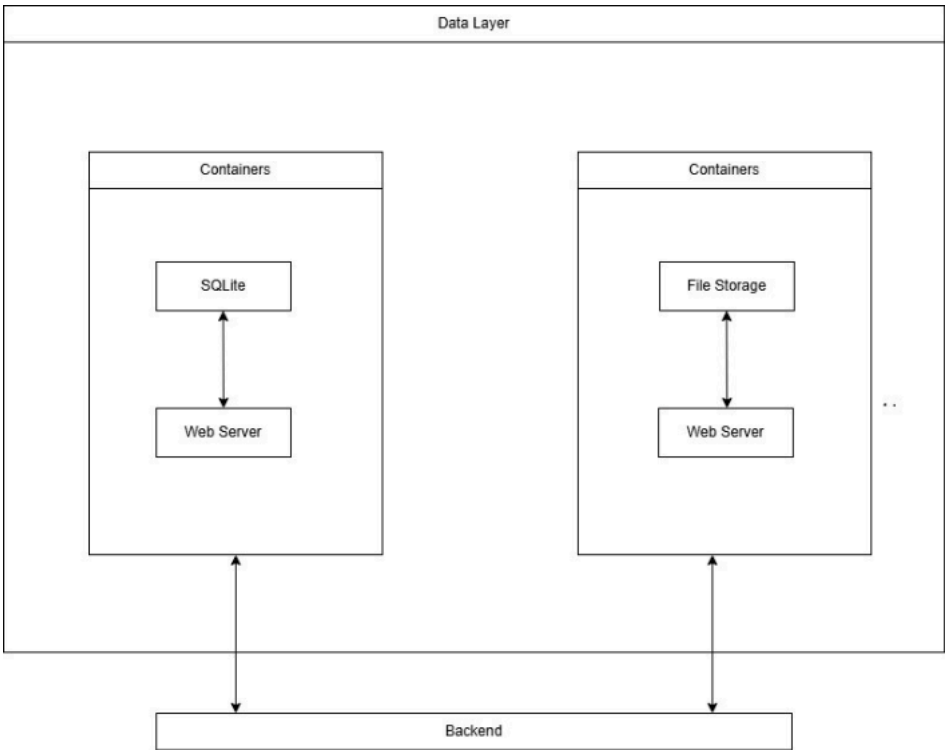
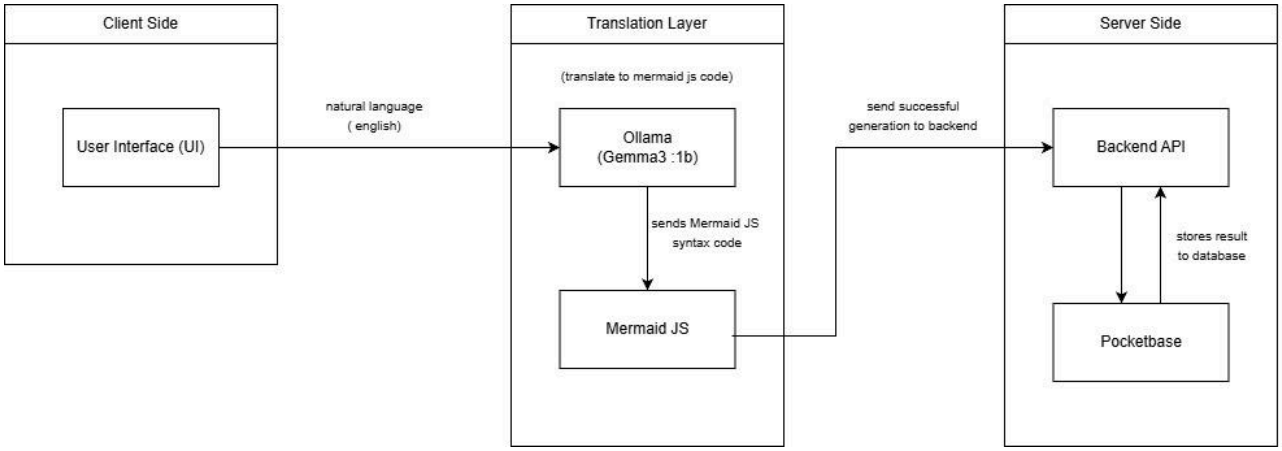


Figure 4.2.1: Data Layer Architecture Diagram

Low Level Architecture

The provided low-level architecture diagram outlines a automated workflow for converting human speech into structured diagrams through a three-tier system consisting of a Client Side, a Translation Layer, and a Server Side. The process begins on the Client Side, where a user interacts with a User Interface (UI) to input a request in natural language (English). This raw text is transmitted to the Translation Layer, which serves as the core processing engine. Within this layer, an instance of Ollama running the Gemma3:1b model interprets the English input and "translates" it into functional Mermaid JS syntax code. Once the code is generated, it is passed to the Mermaid JS component, which validates or prepares the visual representation. Following a successful generation, the data moves to the Server Side for persistence and management. A Backend API receives the generated output from the Translation Layer and facilitates its storage within Pocketbase, a backend-as-a-service solution. This final step ensures that the resulting diagram is saved to a database, allowing it to be retrieved, edited, or viewed by the user at a later time.

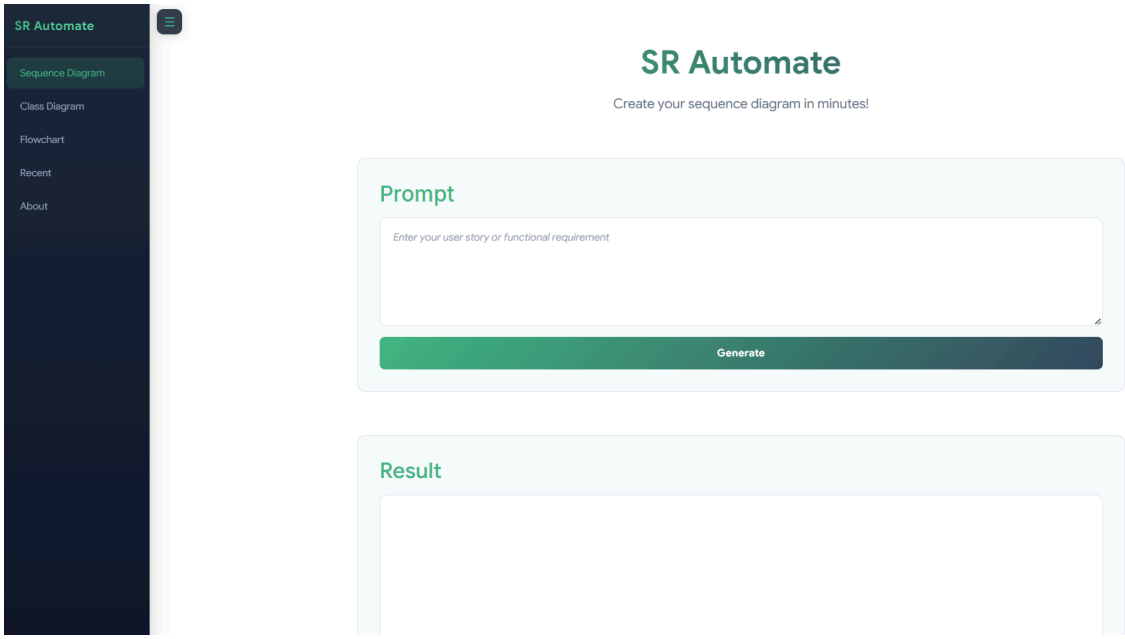


Subsystems Breakdown

Subsystem	Description	Key Components
User Interface (UI)	Handles the visual layout, user input for prompts, and the "Recent Diagrams" gallery dashboard.	Vue.js , Axios
Logic & AI Engine	Manages the communication with the LLM to translate natural language into structured Mermaid syntax	Gemma3 , Ollama
Rendering & Export	Responsible for taking Mermaid code and rendering it into visuals, as well as handling SVG downloads.	Mermaid.js
Data & Persistence	Manages the storage of diagram metadata in the database and physical file storage on the server.	Pocketbase

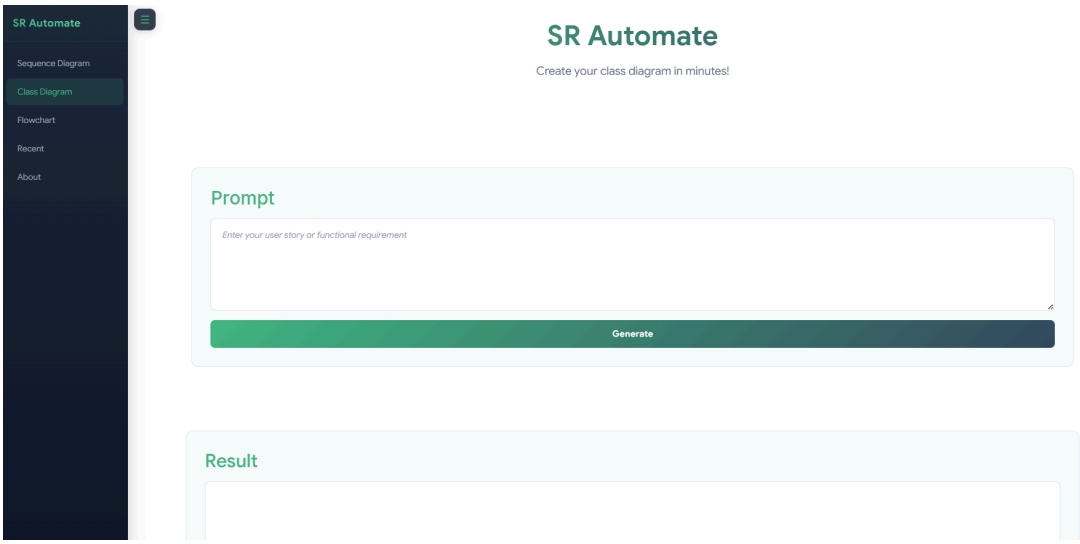
4.3 Main Screens

This is the Sequence Diagram Generation Page. It is used to generate sequence diagram based on user input based on natural language.



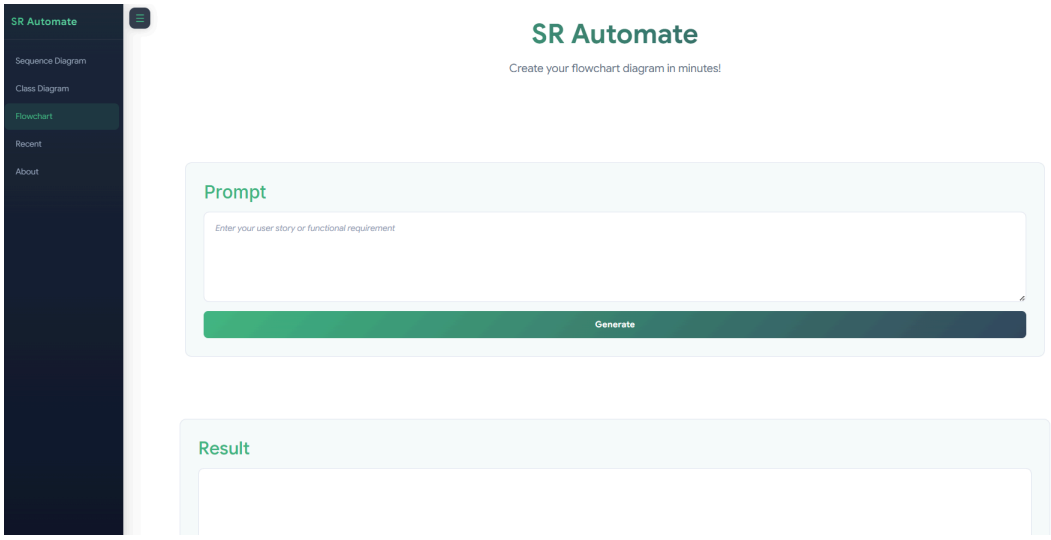
4.3.1 Class Diagram Generation Page

This is the Class Diagram Generation Page. It is used to generate sequence diagram based on user input based on natural language.



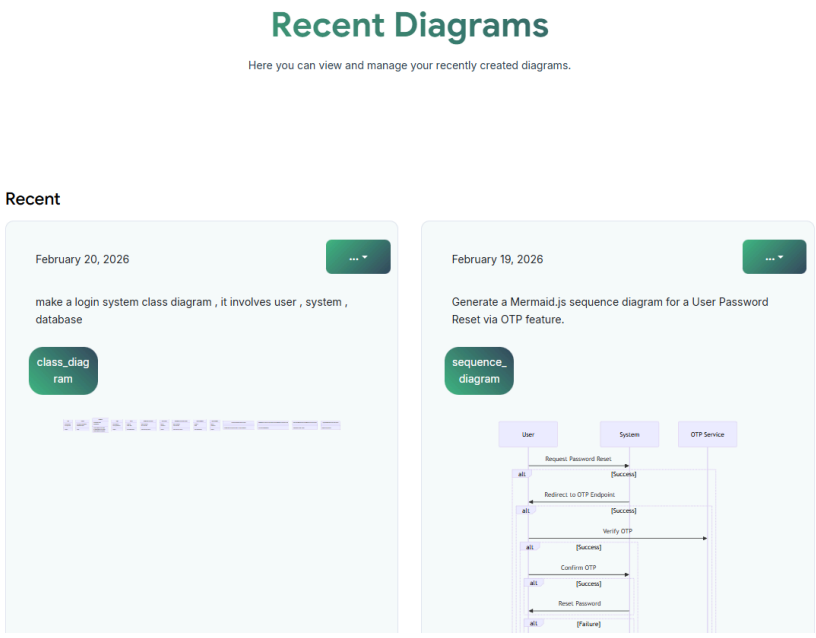
4.3.2 Flowchart Generation Page

This is the Class Diagram Generation Page. It is used to generate sequence diagram based on user input based on natural language.



4.3.3 View Recent Diagrams

This page is used to view recently generated diagram based on successful generation , it consists of diagram prompt , code , diagram image and options to edit , download and delete diagram.



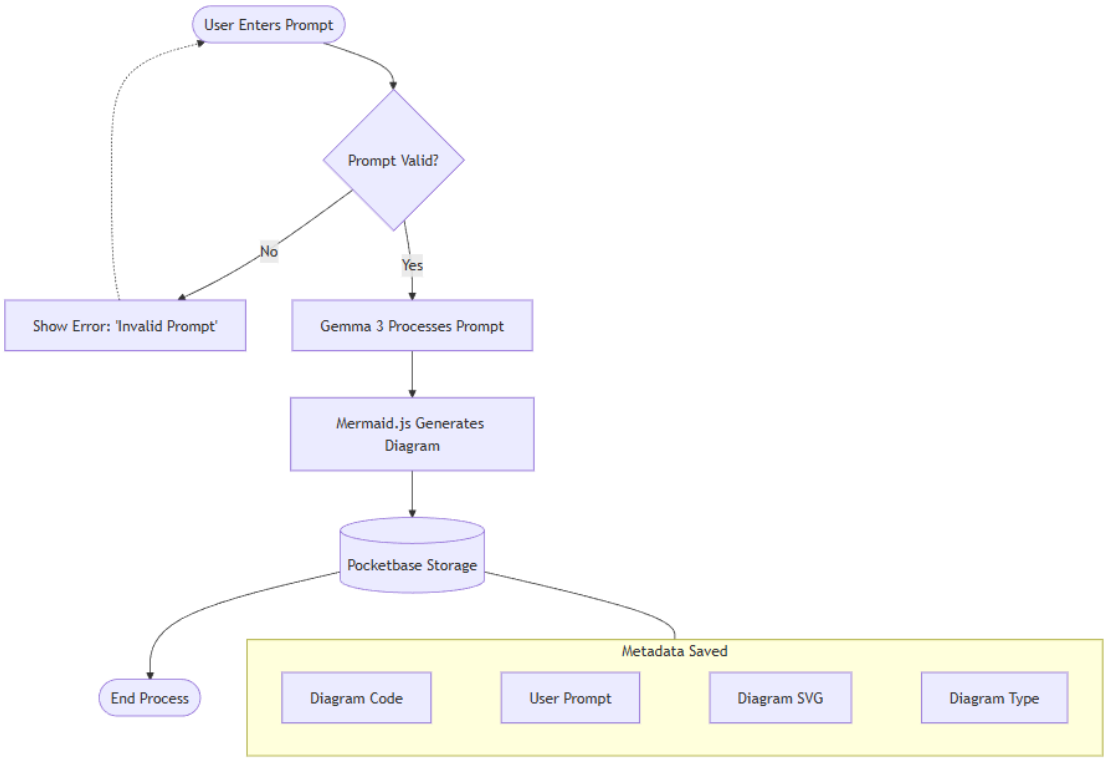
4.4 Main Components

Below are the main components used for this web application. It consists of important parts used to perform operations within this application.

Component	*Subsystem*	*Description*
ClassDiagramFactory	Logic Factory	Contains the systemPrompt, header, and fixer function to sanitize AI output.
generateDiagram	Core Controller	The main async function orchestrating the AI fetch, DOM update, and rendering.
mermaid.run()	Rendering Engine	The Mermaid.js core that parses string-based syntax into interactive SVG elements.
saveDiagramToDB	Data Persistence	Serializes the SVG DOM node and sends it via axios to the backend.
copyImageToClipboard	Export Utility	Uses a hidden canvas and XMLSerializer to convert SVG to high-res PNG.
Ollama (gemma3)	Inference Engine	Local LLM that generates the Mermaid code based on user requirements.
Pocketbase	Backend / DB	Stores the history of generated diagrams, prompts, and SVG files.

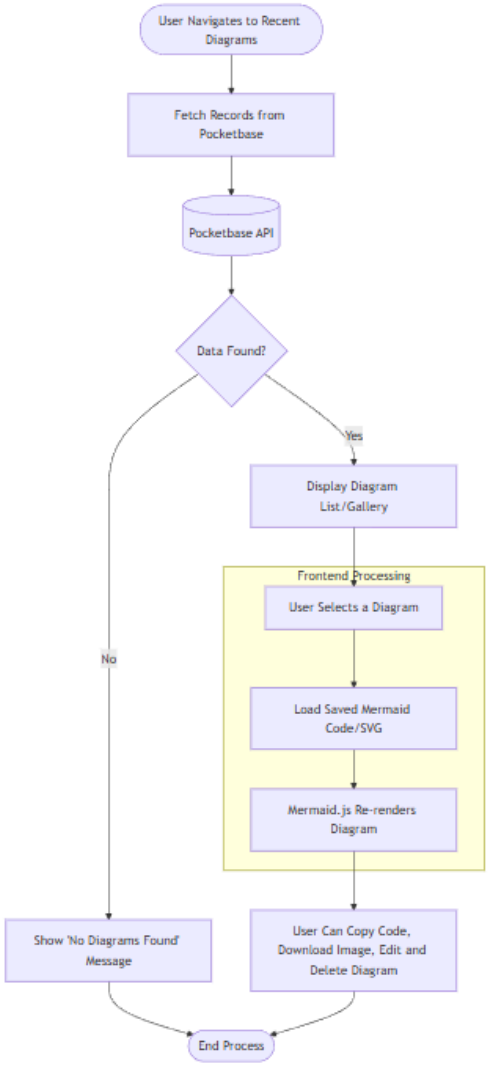
4.4.1 Generate Diagrams

This component explains about Diagram generation based on its types , users will enter natural language prompts and sends it to the system.



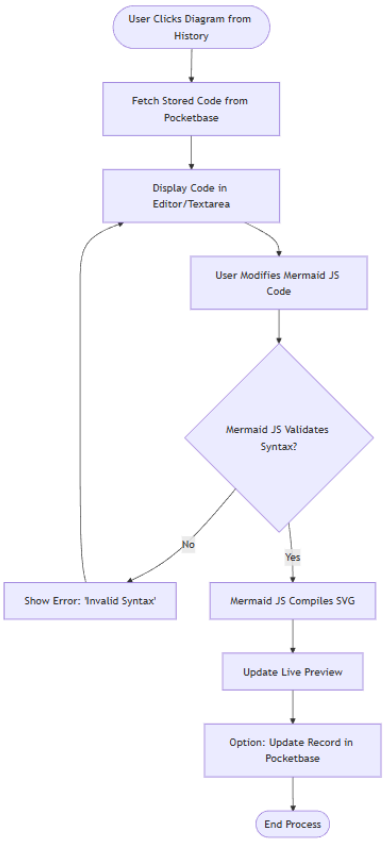
4.4.2 Edit Diagram Code

Users can edit successful generated diagrams based on its types. From there , users can also be able to edit diagram code and having mermaid js to display new and edited diagram based on the code change made by the user.



4.4.3 View Recent Diagrams

Users can view recent successful generated diagrams based on its types. From there , users can also be able to edit , download diagram png , copy diagram code , and delete diagrams.



5 Implementation Details

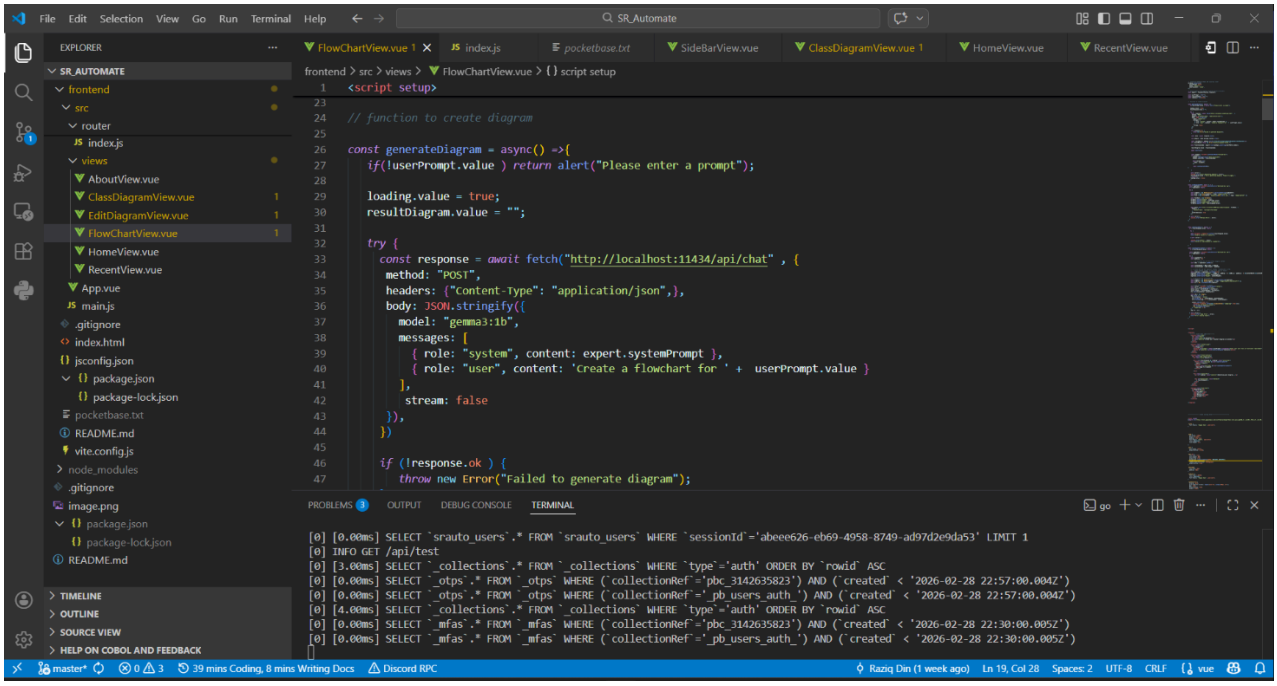
5.1 Development Environment

Our automation system, SR-Automate, is developed using Visual Studio Code as the primary integrated development environment. Visual Studio Code provides a versatile workspace that enhances our development velocity with intelligent code suggestions, seamless extension support, and advanced debugging capabilities tailored for full-stack engineering.

The core of SR-Automate is built on a high-performance backend powered by Go (Golang) and PocketBase. By utilizing PocketBase as a framework, we achieve a highly efficient, "all-in-one" backend solution that combines an embedded SQLite database, real-time subscriptions, and built-in file management into a single portable executable. This architecture ensures rapid development and simplified deployment without the overhead of traditional database management.

To provide intelligent automation and data insights, we integrate Ollama for local Large Language Model (LLM) orchestration. This allows SR-Automate to process complex tasks privately and securely on-device. For system clarity, we utilize Mermaid.js to dynamically generate diagrams and flowcharts from code, ensuring that automated processes are visually transparent and easy to audit.

The frontend is crafted using the Vue.js framework along with modern HTML and CSS. This choice enables a reactive, component-based architecture that delivers a high-performance and user-friendly interface. The combination of these technologies results in a responsive, scalable, and self-hosted automation platform. The image below showcases the Visual Studio Code environment configured for the SR-Automate development lifecycle.



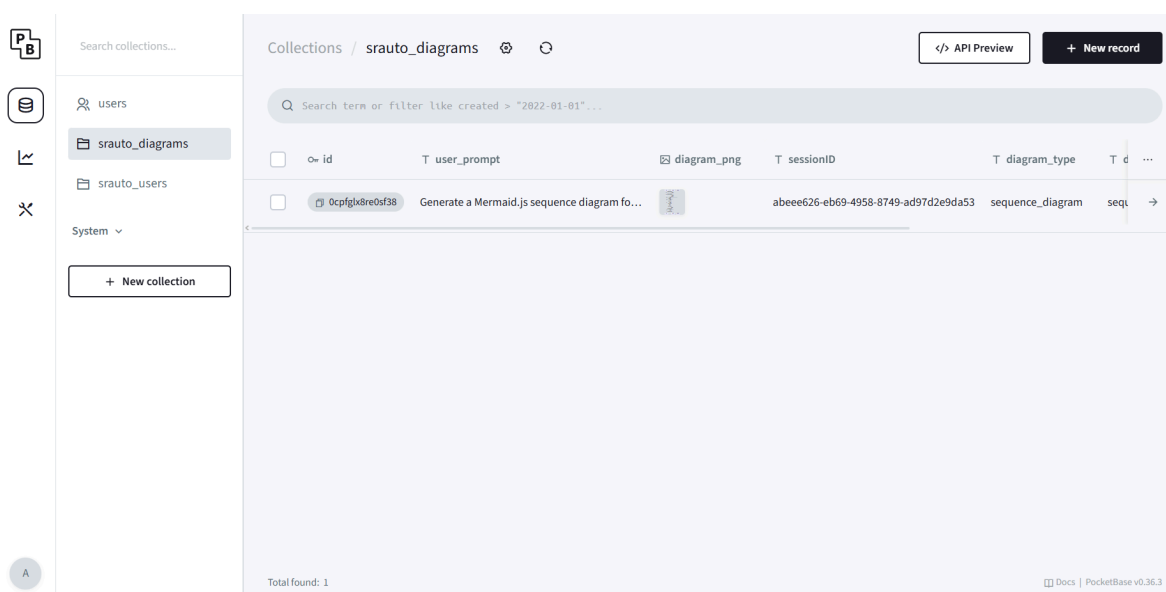
5.2 Software Integration

<*TO DO: Describe a strategy to integrate all the subsystems here with relevant images.*>

File	Description
HomeView.vue , ClassDiagramView.vue , FlowChartView.vue	Generate Diagrams
RecentView.vue	View Recent Successful Generated Diagrams
EditDiagramView.vue	Edit Diagram Code and recompiles it
main.go	The whole backend code with go ecosystem

5.3 Database

For data management and persistence, SR-Automate leverages PocketBase, a modern open-source backend solution built on top of SQLite. Unlike traditional client-server database architectures, PocketBase is embedded directly into our Go backend, providing a high-performance, single-file database that eliminates network latency between the application logic and the storage layer. This integration offers built-in real-time subscriptions, allowing the Vue.js frontend to reflect data changes instantly without manual refreshing. The schema is managed through a user-friendly administrative dashboard, supporting complex relations and automated data validation. Furthermore, the use of SQLite as the underlying engine ensures that SR-Automate remains lightweight and portable, requiring zero external configuration while maintaining full transactional integrity for all automation logs and user configurations.



6 Testing

6.1 Testing Strategy

Our team utilized Top-Down Incremental Integration Testing to develop SR-Automate, prioritizing the user experience and high-level orchestration. We first established the Vue.js frontend architecture and the Go API routing layer, which defined the core interaction flow between the user and the automation engine.

While complex low-level modules, such as Ollama LLM integration, Mermaid.js dynamic rendering logic, and advanced PocketBase hooks were still in development, we employed stubs and mock JSON responses. This allowed us to simulate AI-generated diagrams and database persistence, verifying that the frontend could handle reactive data updates incrementally. As each Go service and AI prompt-template became production-ready, it was integrated into the high-level flow, replacing the stubs to ensure the transition from "mock data" to "live automation" was seamless.

To maintain system integrity throughout this process, the team conducted Daily Smoke Testing. As individual Go modules were updated or Vue components were refactored, we performed rapid executions of critical functions, specifically User Authentication via PocketBase and the initial LLM handshake with Ollama. This routine ensured that the "happy path" of the application remained operational, allowing us to identify and resolve major integration bugs before they could impact the stability of the core automation engine.

6.2 Acceptance Testing

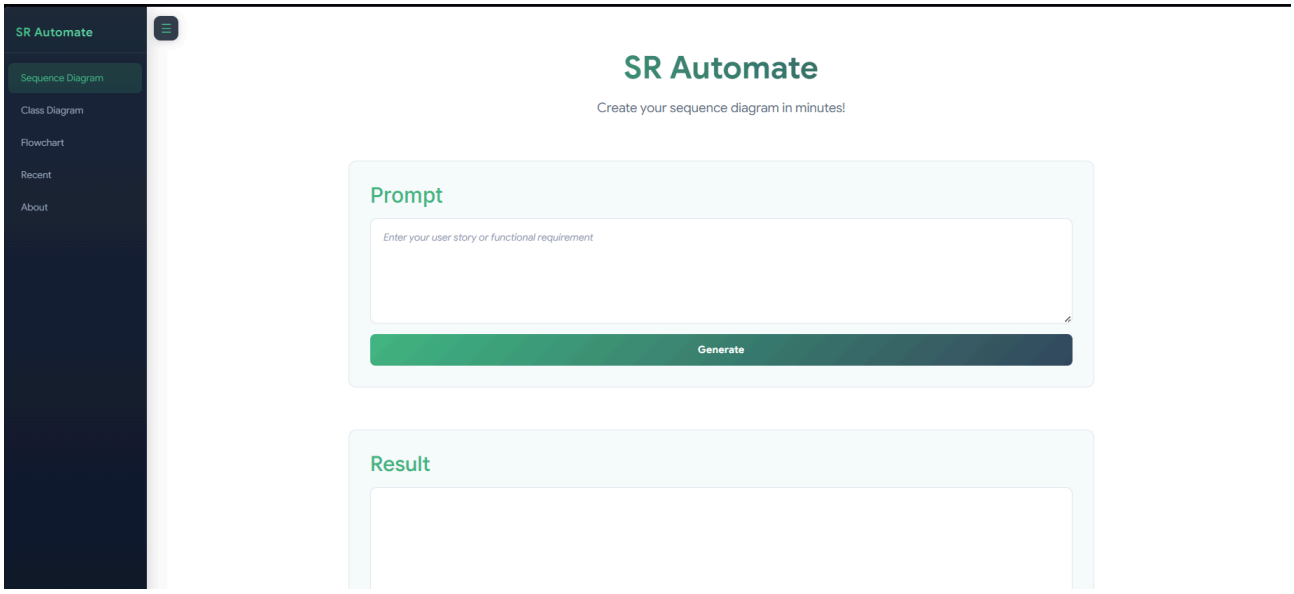
Criteria	Fulfilled	Remarks
Can User enters natural language to prompt	yes	The user input needs to be as general as possible
Can user view the recent diagrams that were generated successfully	yes	The user can view the recent diagrams
Can user edit the diagram based on diagram code syntax	yes	The user needs to know mermaid js syntax to get a valid rendering from mermaid js

Date tested : _1/3/2026_____
% Complete : __1/3/2026_____
Tested by : ____Raziq Din_____
Verified by :____Raziq Din_____
…

7 Sample Screens

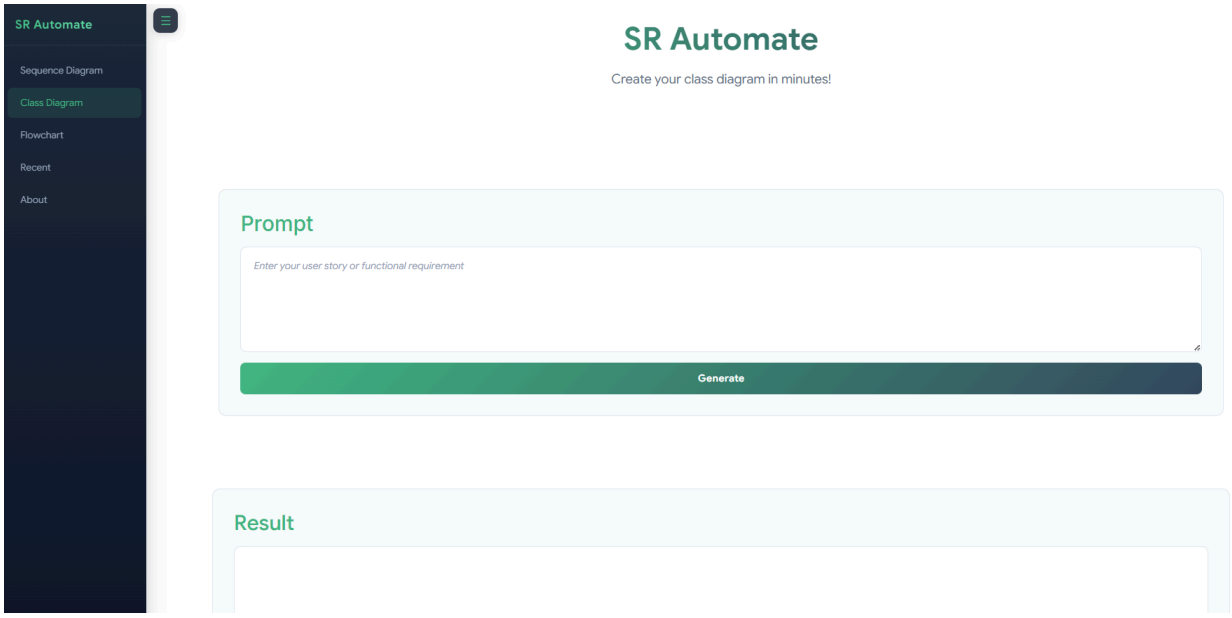
7.1 Sequence Diagram Generation Page

This is the Sequence Diagram Generation Page. It is used to generate sequence diagram based on user input based on natural language.



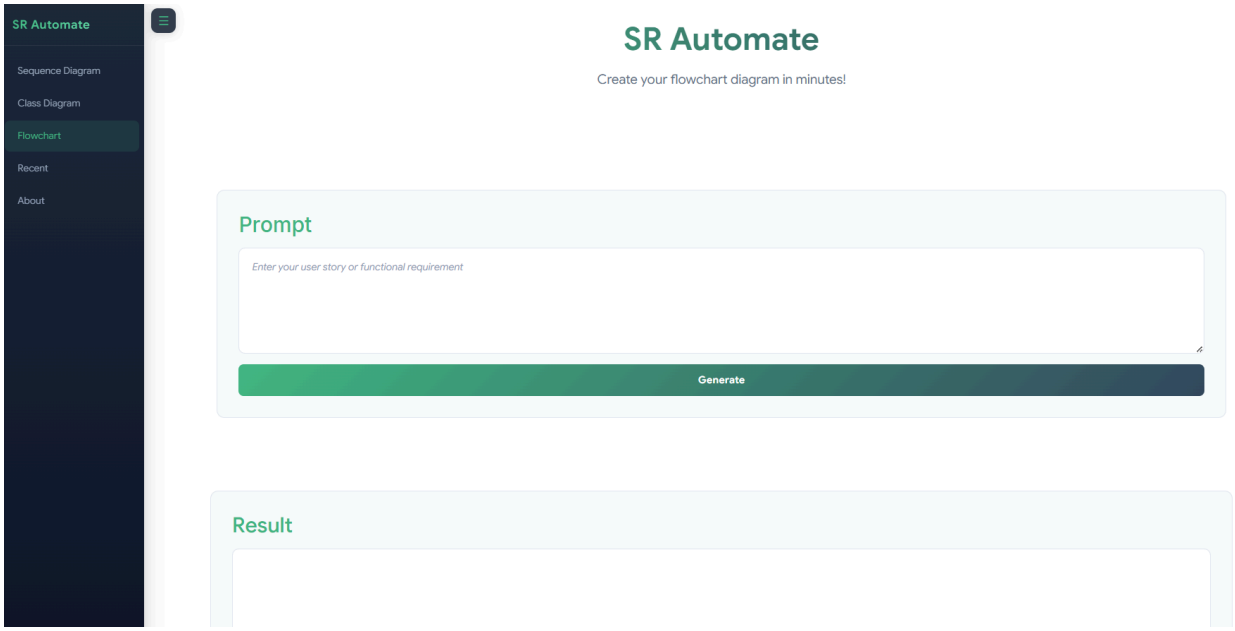
7.1.1 Class Diagram Generation Page

This is the Class Diagram Generation Page. It is used to generate sequence diagram based on user input based on natural language.



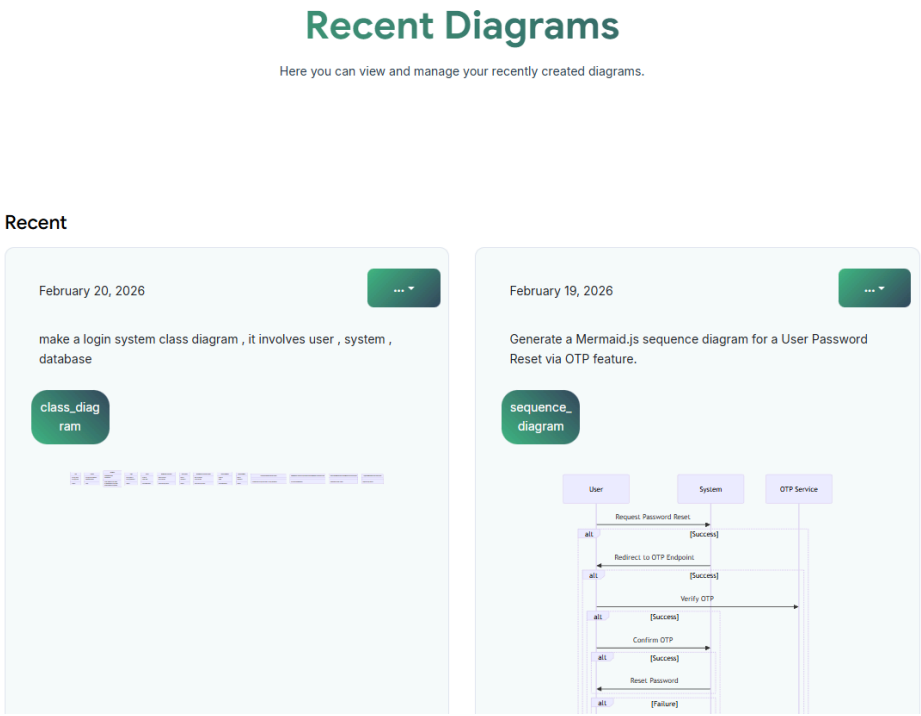
7.1.2 Flowchart Generation Page

This is the Class Diagram Generation Page. It is used to generate sequence diagram based on user input based on natural language.



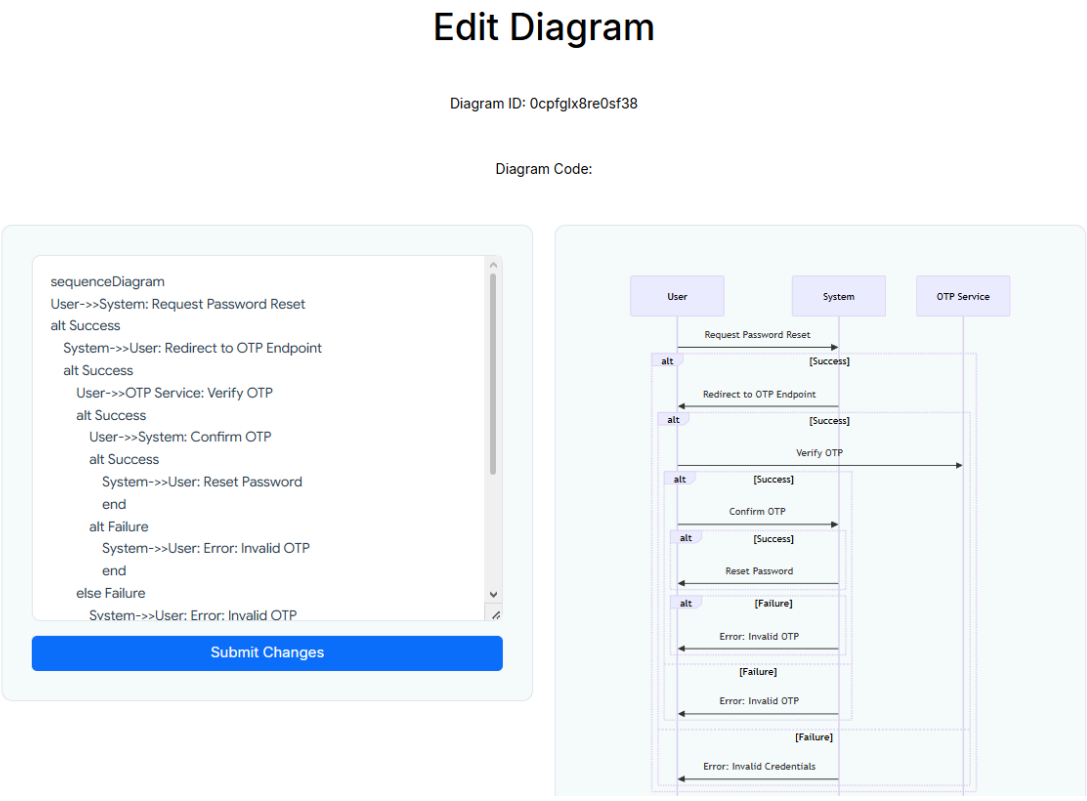
7.1.3 View Recent Diagrams

This page is used to view recent generated diagram based on successful generation , it consists of diagram prompt , code , diagram image and options to edit , download and delete diagram.



7.1.4 Edit Diagram Page

This page is used to edit successfully generated diagram. The user needs to adhere to syntax validity and mermaid js syntax.



8 User Guide

1. Quick Start (Development Mode)

To get the environment running on your local machine, follow these steps:

Clone the Repository:

```
git clone https://github.com/your-username/SR-Automate.git
cd SR-Automate
```

Launch the Development Server:

```
npm run dev
```

1. Note: This command concurrently starts the **Vue.js** frontend and the **Go** backend/PocketBase instance.

9 Conclusion

In conclusion, SR Automate functions as a high-efficiency bridge between conceptual thought and formal documentation, automating the traditionally manual task of system modeling. By integrating a local Gemma 3 LLM with the Mermaid JS engine, the software provides a "Language-to-Diagram" utility that is both private and highly responsive. Its core value lies in its ability to democratize technical drawing, allowing users with no coding knowledge to generate professional-grade flowcharts and schemas through simple English prompts. With a robust Pocketbase backend ensuring that every generated design is persistently stored and easily retrievable, SR Automate serves as a comprehensive management tool for iterative system design and rapid technical prototyping.

References

Github Repository:
https://github.com/nidqija/Software-Requirements-Automate

Go Language Documentation:
https://go.dev/doc/

Vue JS Documentation:
https://vuejs.org/guide/introduction.html

Boostrap 5.3 Documentation:
https://getbootstrap.com/docs/5.3/getting-started/introduction/

Ollama Documentation:
https://vuejs.org/guide/introduction.html

Mermaid JS Documentation:
https://mermaid.js.org/

Gemma3 Documentation:
https://deepmind.google/models/gemma/gemma-3/